ORIGINAL ARTICLE

Integrating multi-camera surveillance with transductive learning for duplicate removal

Jorge Batista · Tatiane N. Rios · Matheus Guimarães · Jorge Nery · Cassio V. S. Prazeres · Rubisley Lemes · Maycon L. Peixoto · Gustavo B. Figueiredo · Frederico A. Durão · Eduardo S. Almeida · Ivan C. Machado · Hérsio M. Iwamoto · Ricardo A. Rios

Received: 11 June 2025 / Accepted: 1 December 2025 / Published online: 17 February 2026
© The Author(s) 2026

Abstract
Video surveillance has benefited greatly from advances in artificial intelligence, particularly in computer vision, and the number of monitored environments has consequently increased, creating new challenges in extracting relevant information. When multiple cameras cover adjacent areas, overlapping fields of view can cause the same subject to be detected across cameras, introducing duplicate counts. Although the literature offers established solutions, many rely on high-quality recordings and complex, computationally intensive methods, limiting their use on resource-constrained devices. In this work, we address these challenges with a solution that leverages minimal information about target subjects and uses a transductive strategy to detect duplicates based on interactions within overlapping fields of view. Experiments in real-world settings show that our approach suppresses duplicates effectively, making it suitable for deployment on resource-limited hardware. We evaluate state-of-the-art lightweight models with high inference speed, as well as classical re-identification methods, in scenarios with low-quality video and constrained devices. The results underscore the effectiveness of the proposed approach and motivate exploration of re-ID with domain adaptation, as well as anchor-free methods with weak or semi-supervised learning.

Keywords Multi-camera surveillance · Transductive learning · Duplicate removal · Deep neural network

1 Introduction

Video surveillance systems consist of cameras specifically configured to monitor activities in both public and private settings while subjects act naturally [1]. The adoption of such systems has been widely increasing to support several monitoring tasks such as security in general [2], accident detection [3], urban mobility [4], personal protection equipments [5], and costumer behaviors [6]. Due to the massive number of recorded scenes continuously monitored, it is imperative to use computer vision techniques to extract useful information automatically.

Another relevant challenge is the integration of multiple cameras in the same area, monitoring the same subjects [7]. In this situation, the recorded scenes may provide different perspectives and possibly duplicated data. Therefore, a relevant research question is the identification of duplicate targets to avoid redundancies and present concise information. To overcome this problem, it is possible to use well-known Deep Neural Network (DNN) models, including face recognition to detect the same person in different cameras as well as person re-identification techniques [8–12]. Besides using DNN, authors have also published related works focused on extracting distinctive invariant features from images to perform reliable matching between different views of objects and scenes [13–15]. These models have proven highly effective across various contexts; however, their performance can be significantly influenced by factors such as infrastructure limitations, video quality, subject positioning, and the distance between subjects and cameras. Furthermore, as emphasized in the literature, key challenges include handling occlusions, varying viewpoints, and changes in appearance over time–issues highlighted by recent studies published in top-tier computer vision conferences and journals over the past two years [16–20]. In such scenarios, beyond the computational cost of image processing, the feature vectors extracted from detected faces may vary substantially, even when representing the same individual.

In this work, we decided to tackle these problems from a different perspective, using fewer details about subjects and creating a semi-supervised approach using transductive learning to detect duplicates according to their interaction in overlapped fields of view captured by cameras. In a nutshell, our approach is based on a primary stage, which runs a supervised method to detect target subjects in multiple scenes recorded from different cameras. Secondly, subjects are mapped as points distributed on a space to be later analyzed by a regression method, whose outputs are used as delimiters to the transition regions. By using delimiters, one can identify overlapped regions and discard multiple detections for the same subjects. There are several situations where this issue occurs, such as counting cars on streets, people in stadiums and shows, animals in wild areas, and more. In this work, we have made some assumptions to assess our proposal carefully: (i) cameras are in fixed positions; (ii) no predefined positions are necessary for the cameras; (iii) high-quality videos are not required; (iv) subjects move freely in the monitored area; and (v) overlapped areas between cameras may exist, potentially showing duplicated subjects. In this context, we collected and analyzed videos recorded indoors with multiple cameras positioned to monitor people in overlapping fields of view.

The contributions of this work can be summarized as follows:

- New approach to detect duplicated subjects in multi-camera monitoring;
- By combining classification and regression methods, fewer subject-specific details are required, eliminating the need for high-quality recordings or complex models to perform face recognition;
- Our contribution is suitable for resource-constrained devices, such as those used in edge computing.

The remaining of this manuscript is organized as follows: we first introduce the problem statement; next, in Sect. 3, we present our proposed approach; Sect. 4 shows the experimental setup and the obtained results; finally, in Sect. 5, we draw conclusions and discuss future work.

2 Case study analysis: removing duplicate users in indoor and outdoor environments

Before providing details about our approach, it is important to detail the environments where the experimental setup was defined. To carefully analyze our results in both qualitative and quantitative ways, we collected recordings from indoor and outdoor scenarios. The indoor one was a computer laboratory, a controlled environment where users (subjects) work and freely move between desks. The space has two blocks (left and right) with monitoring cameras to cover the entire area. As illustrated in Fig. 1, the camera responsible for monitoring the

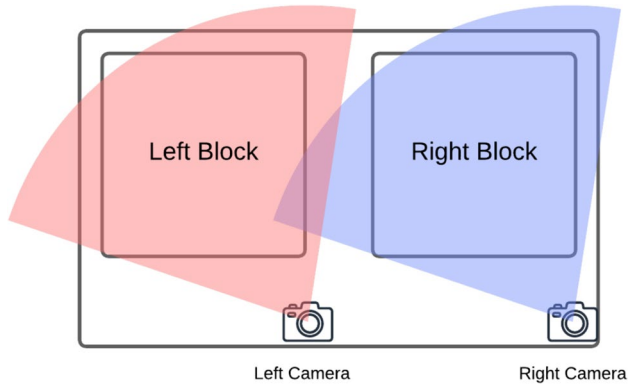


Fig. 1 A simplified top view of the space showing the blocks and the FOV of both cameras. Each camera is pointed at its respective block, but the right one covers an area already covered by the left one

Fig. 2 View of both cameras: left camera on the top and right camera on the bottom. In red, the visual representation of selected edges from a pair of cameras

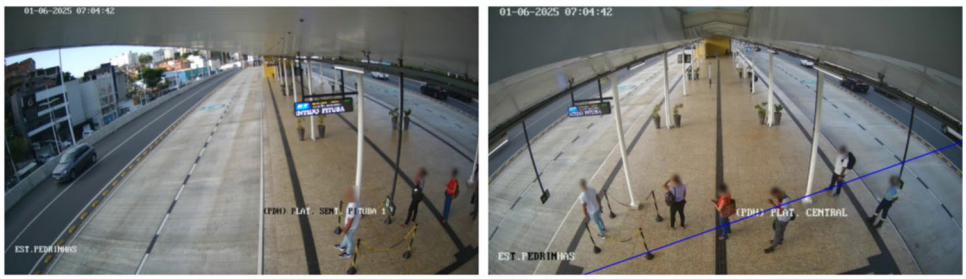


Fig. 3 Passengers in a given BRT station

left block (in red) can capture all users in that region. However, the camera in the right block (in blue) captures not only users in that region but also those from the left one positioned in the overlapped field of view (FOV), as a correspondence problem. As a simple task to show the impact of this issue, by automatically counting subjects using DNN-based models, wrong results are obtained due to the sum of the same users detected in left (correct) and right (duplicated) blocks.

Figure 2 exemplifies our indoor environment, highlighting important challenges tackled by our proposal. Firstly, there is a relevant distance between some subjects and the camera, especially those at the end of the space. Secondly, occlusions may occur when users are partially hidden behind screens or not directly looking at the cameras. These challenges affect the performance of approaches using face recognition to discard the same faces in both cameras.

The second experimental environment focused on removing duplicate users across multiple cameras monitoring passengers at bus stops. This study was conducted in the context of the Bus Rapid Transit (BRT) system in Salvador, Brazil. The BRT system serves approximately 30,000 passengers daily, and currently operates with around 40 buses distributed across three lines and 20 stations. The decision-making process for scheduling bus departures relies on counting the number of passengers at each station. As illustrated in Fig. 3, this counting is

influenced by data from multiple cameras. Following the validation of our proposal in an indoor environment, we deployed it in a more complex setting to support the BRT system's operations.

3 Proposed approach

This section describes our proposed approach, which consists of two main stages, as detailed in Algorithm 1. In the first stage, we train a supervised model to detect the target subjects, i.e., users who move freely through indoor and outdoor environments, specifically a computer laboratory and bus stations. The second stage focuses on automatically generating delimiters that highlight transition regions where user duplication is likely to occur.

Along with our contribution, we also present in this section a strategy using a DNN model trained to crop regions in images using predefined bounding boxes previously defined by specialists, thus explicitly removing overlapped fields of view. Our approach does not directly use this strategy; it was implemented as a baseline in our comparison.

3.1 First stage—face detection

The detection used in this stage depends on the problem under investigation (e.g., objects, cars, or animals) and can be modified according to the readers' needs. Once the focus of our investigation is on users, we could consider both person or face detection. We opted for face detection in this approach due to the presence of numerous objects in the environment that often occlude people, but leaving mostly their faces visible. All faces in the collected videos were manually annotated, resulting in 2,414 bounding boxes—approximately 600 faces per video. Next, we employed YOLO (You Only Look Once) [21] version 11, from the Ultralytics library, in our experiments. Table 1 summarizes the key architectural parameters of this model.

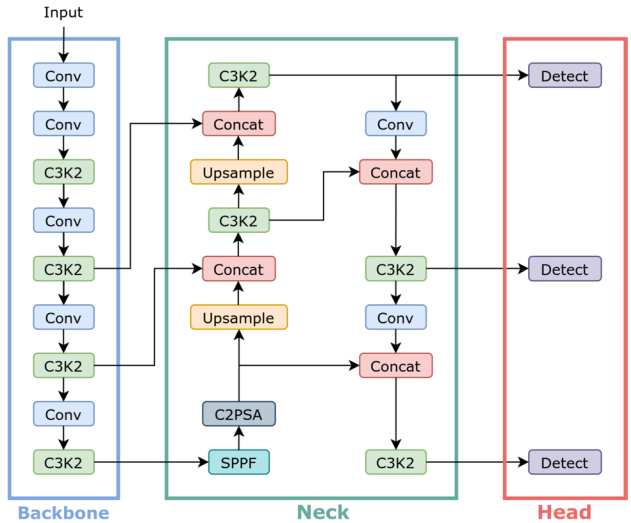
YOLO is a family of real-time object detection models known for treating detection as a single regression problem, directly predicting bounding boxes and class probabilities from full images in one evaluation. This unified and efficient approach makes YOLO particularly suitable for tasks requiring high-speed inference. As shown in Fig. 4, YOLO models are generally organized into three major components: backbone, neck, and head.

The backbone, highlighted in blue in Fig. 4 (Backbone), refers to the portion of the network responsible for extracting general visual features from the input image. These features encode information about edges, textures, shapes, and more complex patterns as the network deepens. In YOLOv11, the backbone employs a module known as C3k2, which builds upon the Cross Stage Partial (CSP) design and uses small convolutional kernels to reduce computational load while preserving expressive power.

Table 1 Summary of key architectural parameters of the YOLOv11 model

Attribute	Value	Description
Input Shape	[B, 3, 640, 640]	*B*: batch size; 3: number of RGB channels; 640×640: input image resolution in pixels
Output Shape	[B, 5, 8400]	*B*: batch size; 5: bounding box attributes (x, y, w, h, objectness); 8400: number of predictions per image
Number of Layers	181	Total number of layers, including convolutional, normalization, and activation layers
Parameters	2,590,035	Total number of trainable parameters in the model
GFLOPs	6.4	Giga Floating Point Operations per image during a forward pass with 640×640 input

Fig. 4 YOLOv11 architecture



The neck, shown in green in Fig. 4 (Neck), is responsible for refining and aggregating the features extracted by the backbone to enhance the model's ability to detect objects at different scales. It typically aggregates information from different layers or scales to improve the model's capacity to detect objects of varying sizes and contexts. YOLOv11 uses the Spatial Pyramid Pooling – Fast (SPPF) module in this stage. The SPPF pools features across multiple receptive fields, enhancing the network's ability to localize small or partially occluded objects.

Finally, the head, illustrated by the red box in Fig. 4 (Neck), is the task-specific part of the architecture that generates the final predictions. In object detection, this includes estimating bounding box coordinates, objectness scores, and class probabilities. YOLOv11's head incorporates a Convolutional block with Parallel Spatial Attention (C2PSA), which improves the model's focus on the most relevant spatial areas in the image, thus refining detection performance across different tasks, including object detection, segmentation, and pose estimation. The full structure of YOLOv11, including the backbone, the neck, and the head components, is illustrated in Fig. 4.

As discussed later, YOLOv11 is available in several variants (yolo11n, yolo11s, yolo11m, yolo11l, and yolo11x) each offering different trade-offs between performance and computational cost. According to benchmark reports from Ultralytics, the yolo11m variant achieves a mean Average Precision (mAP) of 51.5% on the COCO dataset while utilizing approximately 22% fewer parameters than YOLOv8m. This reduction in model size, coupled with competitive performance, positions YOLOv11 as a promising candidate for deployment in both resource-constrained and high-performance environments.

3.2 Second stage—semi-supervised field of view projection estimation

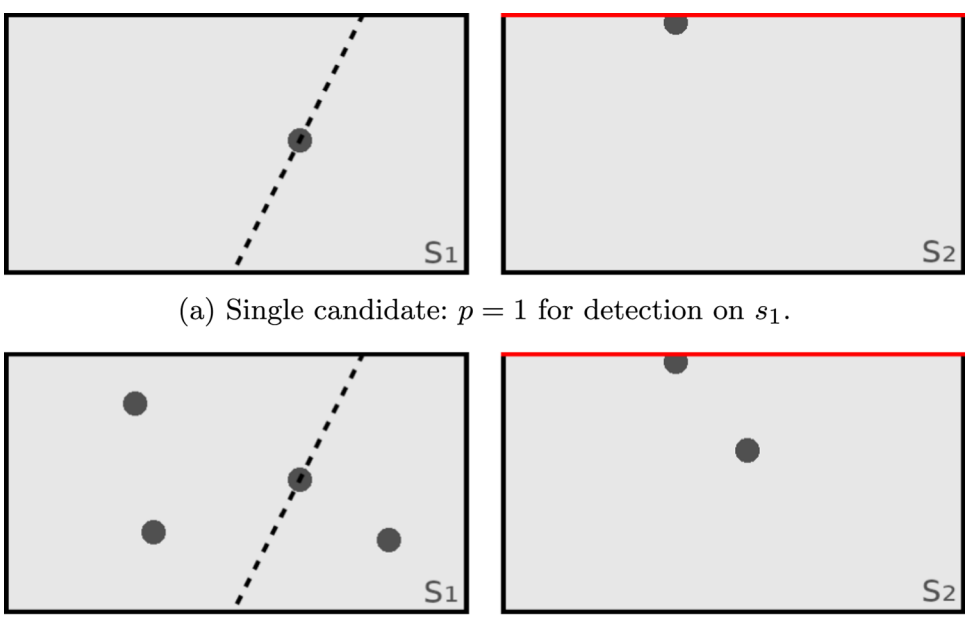
As previously discussed, the proposed approach begins by aligning frames from different cameras and detecting bounding boxes around each identified face to serve as coordinate references (Algorithm 1—Lines 1 to 5). Next, the coordinates are modeled by regressors to create delimiters for the transition regions between fields of view (FOV) monitored by a set of surveillance cameras $S = \{s_1, s_2, \ldots, s_k\}$. Hence, considering a scenario with two cameras as stated in our case study, FOV for a camera s_1 is projected to recordings from s_2, and vice versa. In summary, each delimiter works as a separator to avoid repeatedly considering the same subject in different cameras, i.e., the same target subject can be detected in multiple cameras; however, it is analyzed only once.

In practice, the subject detection in overlapping regions can present some imprecision caused by the deviations in the FOV lines or the parallax effect when the subject appears to have a different position relative to the background between the views of each camera. To attenuate this problem, in our case study, a good estimator for the number of users (U) can be obtained by $U = \sum_{i=1}^{n} D_i - E[O]$, in which $D = \{d_1, d_2, \ldots, d_l\}$ and $O = \{o_1, o_2, \ldots, o_m\}$ are the total number of detected faces and detected faces inside the overlapping region for all cameras in S. If the analysis is computed between pairs of cameras and the presence of users inside their FOV has the same probability, the number of users is estimated by the difference between the sum of the difference detected and the expected users in FOV.

3.2.1 Initial setup and finding matches

Considering our case study, the red edges in the left and right blocks, illustrated in Fig. 2, define how the cameras must be integrated into our proposed approach. To estimate the FOV delimiter on Camera s_1, we monitor Camera s_2, looking for detections of people entering the frame of Camera s_2 through one of its red edges (Algorithm 1—Line 6). When this happens, the person detected entering the frame of camera s_2 should also be present in the frame of camera s_1, approximately along the FOV delimiter line of Camera s_1 that we want to estimate. Therefore, we want to record the coordinates of this person on Camera s_1 to use this information in the estimation process (Algorithm 1—Lines 7 to 11).

In the best-case scenario, when a person is detected entering the frame of Camera s_2 through one of its red edges, there will only be one person present in Camera s_1, allowing us to confidently identify them as the same individual (Fig. 5a). However, in typical conditions, there may be multiple people present in Camera s_1 (Fig. 5b). Since no re-identification technique is employed, it is impossible to determine which individual corresponds to the person detected in Camera s_2, leading to ambiguity. Thus, when a person enters the frame of Camera s_2 through one of its red edges, we record the coordinates of each person detected in Camera s_1 as a possible match,



(a) Single candidate: $p = 1$ for detection on s_1.

(b) Four candidates: $p = 0.25$ for detections on s_1.

Fig. 5 Two example scenarios for matching when a detection occurs on an selected edge in s_2

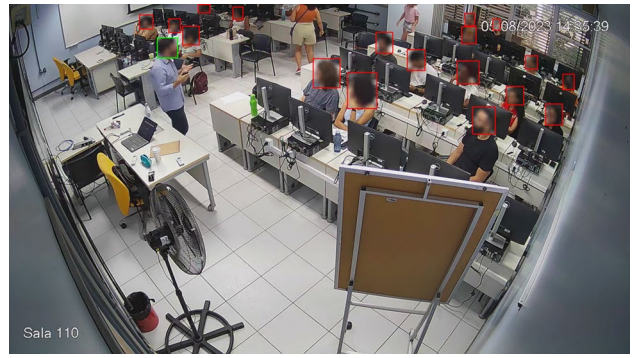


Fig. 6 Movement status of tracked instances. Moving instances in green, stationary instances in red

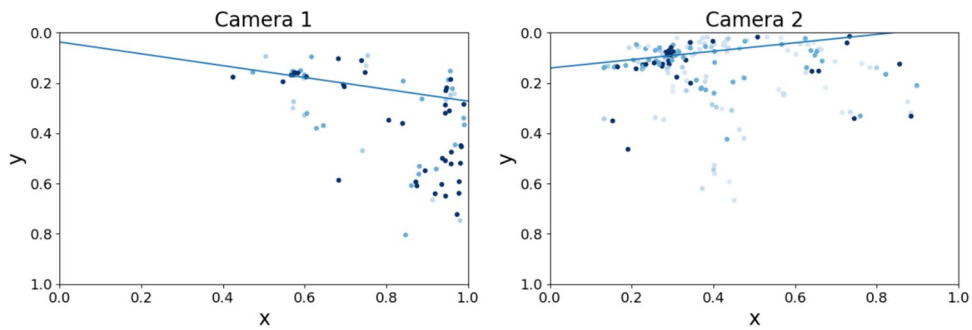


Fig. 7 FOV lines estimated through RANSAC regression

along with an attribute $p = \frac{1}{j}$ for each set of coordinates, with j being a given number of detections present in s_1, to represent the probability of each one of these detections being a true match.

As a person crossing a red edge to appear in the image of Camera s_2 is in motion, the candidate detections for matching in s_1 must also be moving. Through tracking, we can identify the detected instances in s_1 that are nearly stationary and eliminate them as potential matches, excluding them from the probability calculations for the remaining detections. An example of this situation is shown in Fig. 6.

To estimate the FOV delimiter on Camera s_2, we follow the same procedure, recording possible matches in s_2 for any person entering the frame of s_1 through one of its red edges (Algorithm 1—Lines 13 to 18). After processing the frames of both cameras, we end up with a collection of possible matches for each camera, as illustrated by the blue-scaled dots in Fig. 7.

3.2.2 Weighted RAMSAC regression

Lastly, we execute the regression method called Weighted Random Sample Consensus (wRANSAC) [22] on a set of possible matches collected from the previous step (Algorithm 1—Lines 21 and 22). This regression is a robust technique used to model datasets with a significant number of outliers. To best fit inliers, RANSAC runs an interactive process that randomly samples a subset of data to train a classification model. Next, the classifier predicts the remaining data as inliers and outliers. The process is interactively repeated, keeping the final model with the greatest number of inliers. An important contribution of our work is to set weights to all data with the

probabilities obtained for each possible match. In Fig. 7, we illustrate the execution of wRANSAC on a set with possible matches collected from two cameras in our case study.

Algorithm 1 Semi-supervised Field-of-View (FOV) Delimiter Estimation

```
Input: Synchronized video streams captured by camera objects s_1, s_2
Output: Estimated FOV delimiter lines for each camera
Data:
s_1: Camera 1
s_2: Camera 2
detections_s1 ← empty list;
detections_s2 ← empty list;
foreach synchronized frame pair (f_1, f_2) from (s_1, s_2) do
    faces_s1 ← YOLOv11(f_1);
    faces_s2 ← YOLOv11(f_2);
    foreach face entering s_2 through red edge do
        candidates_s1 ← MovingFaces(faces_s1);
        j ← length(candidates_s1);
        foreach candidate in candidates_s1 do
            detections_s1.append((candidate.coordinates, 1/j));
        end
    end
    foreach face entering s_1 through red edge do
        candidates_s2 ← MovingFaces(faces_s2);
        j ← length(candidates_s2);
        foreach candidate in candidates_s2 do
            detections_s2.append((candidate.coordinates, 1/j));
        end
    end
end
FOV_line_s1 ← wRANSAC(detections_s1);
FOV_line_s2 ← wRANSAC(detections_s2);
return FOV_line_s1, FOV_line_s2
```

3.3 Supervised baseline: area detection

According to the literature, in addition to face recognition, some authors address this problem using overlapping area detection. In our experiments, the proposed approach was evaluated by integrating the recorded areas from multiple cameras using Deep Neural Networks (DNNs). Similar to the face detection stage, we created bounding boxes to highlight regions of interest (ROIs) in all images from the right camera block, as illustrated in Fig. 8. We then trained a DNN to identify similar ROIs in new video frames and to discard any faces detected outside these regions–assumed to be duplicates originating from the left camera block.

The main advantage of this approach is its simplicity, which makes it easily implemented in different contexts and infrastructures. On the other hand, its main disadvantages are the requirement for a specific ROI defined in advance for all application and the need to redefine and retrain the ROI when the camera position changes.

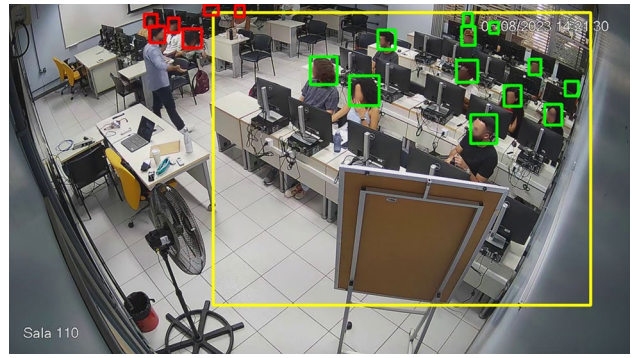


Fig. 8 Example of area detection—the right block detection represented by the yellow box. Red and green bounding boxes representing duplicated and correct faces, respectively

4 Results

The analyses performed to assess the proposal followed conventional practices in the field of Computer Vision (CV) for object detection tasks, which is based on the Intersect over Union (IoU) ratio, $IoU = \frac{G \cap P}{G \cup P}$. It essentially computes the performance by comparing the overlapped areas between an expected (ground truth) bounding box (G) and the predicted one (P).

During inference, the confidence C of a bounding box is defined by $C = Pr(Class_i|Object) * Pr(Object)$. By choosing a threshold $\tau_1 \in [0, 1]$ for $C \geq \tau_1$, we decide whether we should consider the predicted bounding box as a detected object from Class *i*, $i = 1...N$. By choosing a threshold $\tau_2 \in [0, 1]$ for $\text{IoU} \geq \tau_2$, we can determine whether there is sufficient overlap between a predicted bounding box and a ground truth box of the same class to consider the detection a True Positive (TP). If a predicted bounding box does not sufficiently overlap any ground truth box (i.e., $\text{IoU} < \tau_2$), it is counted as a False Positive (FP). Conversely, if a ground truth object is not matched by any prediction with $\text{IoU} \geq \tau_2$, it is considered a False Negative (FN).

After calculating the number of TP, FN, and FP, we can derive important measures such as Precision, Recall, and F1-Score. In summary, Precision assesses the exactitude of the model by computing the rate of correctly predicted detections over the number of predicted detections, while Recall evaluates the completeness of the model by computing the rate of correctly predicted detections over the number of elements expected to be detected. F1-Score represents a harmonic mean of Precision and Recall. The Average Precision (AP) is a metric that combines Precision and Recall values over a range of τ_1, given a fixed τ_2. Two other metrics, based on AP, that are widely used to compare performance between models are mAP@50 and mAP@50-95. The mAP@50 (mean Average Precision at IoU = 0.50) considers a detection correct if the IoU is at least 0.50, providing a smoother evaluation. In contrast, mAP@50-95 is a more comprehensive metric that averages AP across multiple IoU thresholds, from 0.50 to 0.95 in steps of 0.05, offering a more stringent and balanced assessment of detection accuracy.

We started our experiments by evaluating different pre-trained YOLOv11 architectures (yolo11n, yolo11s, yolo11m, yolo11l, and yolo11x) on a Raspberry Pi 4, a limited device usually considered in Edge Computing applications. The first analysis was focused on assessing the overall performance of such models in detecting labeled faces in our dataset. According to the curve F1-score × confidence (Fig. 9a), all models presented great performance varying from 0.87 to 0.91, showing no significant difference between them. A similar behavior was noticed in the Precision-Recall curve shown in Fig. 9b.

Next, we analyzed such models in terms of mean inference time, memory usage, and CPU consumption. According to the results presented in Table 2, yolo11n demonstrated the fastest mean inference time (0.507 s) with a memory footprint of 345 MB and a CPU usage of 319%. Given our objective of running our proposal on resource-constrained hardware, we selected yolo11n to proceed with our experiments.

In the next phase of our investigation, we fine-tuned the yolo11n model using images from our dataset. To ensure robust and generalizable results, we adopted a 4-fold cross-validation strategy, where each fold comprised

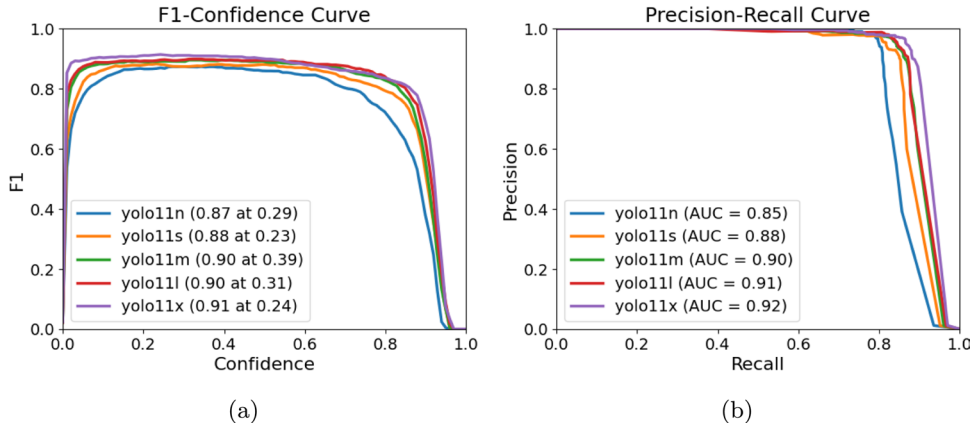


Fig. 9 **a** F1-confidence curves and **b** precision-recall curves for five YOLO models evaluated on the validation dataset

Table 2 Comparison of YOLOv11 Architectures on Raspberry Pi 4

Architecture	Time (s)	Memory (MB)	CPU (%)
yolo11n	0.507	345	319
yolo11s	1.118	505	360
yolo11m	4.823	651	379
yolo11l	5.707	848	379
yolo11x	16.212	1191	384

Table 3 Training results after fine-tuning the model with our dataset

Fold	τ_1	F1-Score	Precision	Recall	mAP50	mAP50-95
1	0.373	0.932	0.950	0.915	0.967	0.587
2	0.182	0.930	0.951	0.911	0.965	0.607
3	0.496	0.889	0.890	0.889	0.958	0.649
4	0.487	0.924	0.953	0.897	0.957	0.481

videos from a distinct location. This setup allowed for a balanced trade-off between having a sufficient number of images for both training and testing while maintaining diversity across folds.

Each fold was trained with a batch size equal to 16, an optimizer based on a stochastic gradient descent method (AdamW algorithm), and a learning rate equal to 0.01 for 100 epochs. After completing the training, we selected the value of τ_1 that showed the highest F1-Score for each fold. For F1-Score, Precision, and Recall, a $\tau_2 = 0.7$ was empirically considered to maximize the results. As shown in Table 4, the final model exhibited a substantial improvement in detection performance following fine-tuning with our dataset. Notably, yolo11n was approximately 30 times faster than yolo11x (16.212s vs. 0.507s, as shown in Table 2) and achieved a higher F1-score after fine-tuning (Table 3) than all versions without fine-tuning (Fig. 8).

While the previous results demonstrate the feasibility of deploying YOLOv11n on resource-constrained devices, we also replicated the experiments using NanoDet [23], a fully convolutional, one-stage, anchor-free object detector. NanoDet's main advantage is its lightweight design and high inference speed, which make it well suited for embedded and mobile scenarios. Table 4 reports NanoDet's performance across the same cross-validation folds used for YOLOv11n, enabling a direct comparison under identical conditions.

In our experimental setup, YOLOv11n achieved considerably better results. Given its favorable CPU utilization, memory footprint, and inference time on the Raspberry Pi 4, we selected YOLOv11n for the subsequent evaluation stages. Nonetheless, based on our findings, NanoDet remains a highly suitable alternative, especially for deployments on more constrained devices.

The experiments designed to evaluate our contribution followed a four-step methodology: (i) users were manually labeled in both camera views to establish the ground truth; (ii) a face recognition model was assessed to

Table 4 Results for the NanoDet model

Fold	τ_1	F1-Score	Precision	Recall	mAP50	mAP50-95
1	0.517	0.525	0.660	0.436	0.853	0.374
2	0.521	0.708	0.803	0.633	0.906	0.489
3	0.462	0.777	0.817	0.741	0.959	0.540
4	0.456	0.671	0.771	0.594	0.906	0.491

(a) (b)

Fig. 10 Users monitored by two different cameras: **a** left view and **b** right view. This experiment was conducted to evaluate the effectiveness of face recognition in detecting duplicate user representations across cameras

determine its ability to identify the same individual across different cameras; (iii) a baseline model based on area detection, commonly used in the literature on correspondence problems, was executed; and (iv) our proposed approach was applied to automatically integrate scenes and eliminate duplicate user representations.

Figure 10 illustrates a teaching scenario in which two cameras monitor the same space with a partially overlapping field of view. Our objective is to evaluate whether students captured by the left camera, Fig. 10a, can be accurately recognized by the right camera, Fig. 10b.

The experiment began by detecting all faces in both camera views. Subsequently, each detected face was converted into an embedding vector using the VGG-Face model as a feature extractor. In the next step, embedding vectors from the left camera were compared to those from the right camera using the Euclidean distance metric, as commonly adopted in the literature. The VGG-Face model considered in our experiments was accessed through the DeepFace library [24]. Figure 11 presents the distances between pairs of faces. Due to space constraints, only the 10 closest matches are highlighted. Given the distance between the subjects and the cameras, none of the faces captured by the left camera were correctly matched with those from the right camera. This outcome is expected, as the same faces are captured from different angles by the two cameras, which significantly affects the similarity between their corresponding embeddings. This experiment underscores the limitations of using classical face recognition as a re-identification (re-ID) method in settings with low-quality cameras or when subjects are far from the sensors. Even in the absence of occlusions, small face sizes, pose and illumination changes, and motion blur reduce the discriminative content of the extracted embeddings, leading to low match rates and failure to correctly resolve multiple entries.

Given the insufficient results previously presented, we investigated the feasibility of removing duplicates by employing an area detection model for people counting. Figure 12 shows the actual and estimated numbers of users, represented by the yellow and blue lines, respectively. As one may notice, this approach effectively produces results that align closely with the expected behavior. It is important to note that this model detects regions of interest that are manually defined during the training phase. Its main limitation, however, is the need to manually annotate the overlapping regions whenever the monitored scenario changes, which in turn requires retraining the detection model.

In the second experiment, we assessed the performance of our approach by automatically detecting transition regions between fields of view (FOV). The obtained results are presented in Fig. 13. By visually analyzing these plots, it is possible to notice that our approach outperforms the area detection in Fold 1 and presents similar results to the remaining ones.

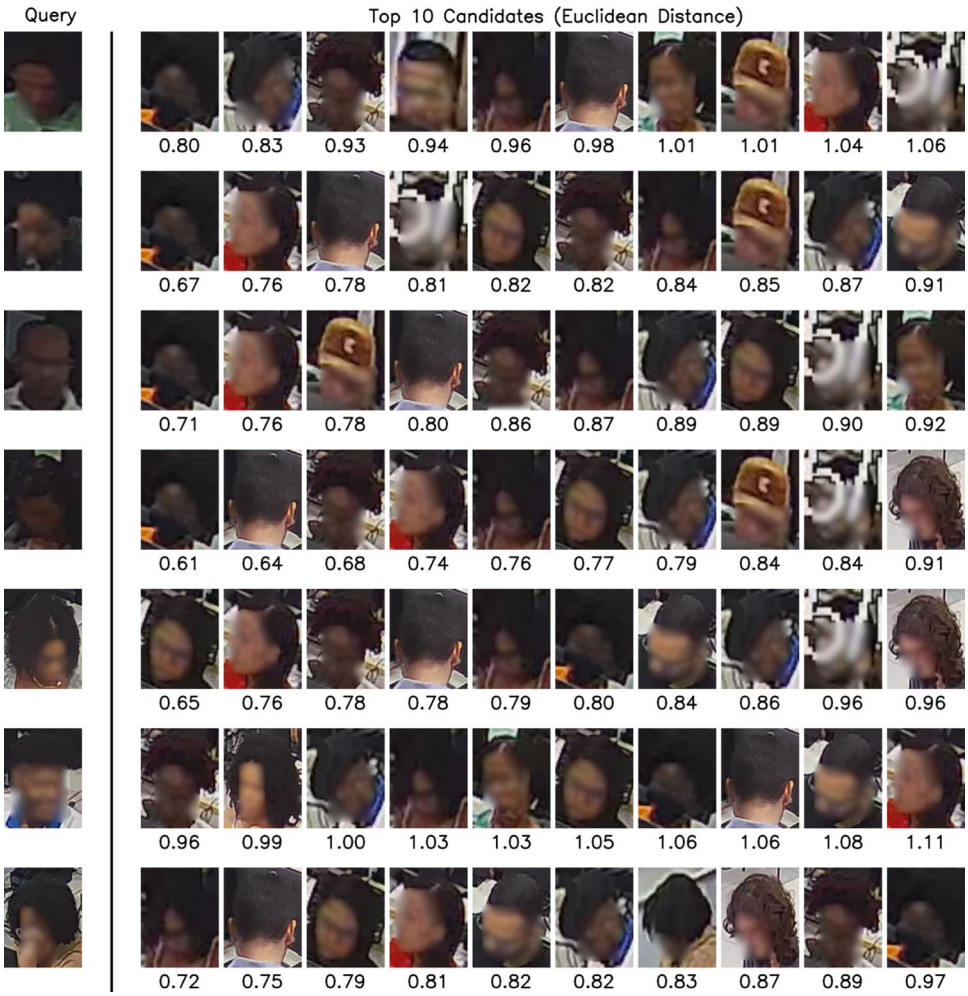


Fig. 11 Analysis of corresponding faces captured by the left and right cameras. The plot shows pairwise distances between face embeddings, with only the 10 closest matches highlighted due to space constraints. No correct matches were found, likely due to the distance between subjects and the cameras

An additional evaluation was conducted using the estimated FOV lines to perform the counting on the manually annotated users instead of the detections generated by the face detection model used in our proposal. In summary, we want to determine whether errors in counting users were caused by the automatic detection of lines or by user detection errors. Results presented in Fig. 14 emphasize that some errors are indeed caused by the model used to detect users, suggesting that our approach may provide superior results with an improved version of the user detection model.

Finally, in addition to visually analyzing the results, we employed the following regression validation metrics to evaluate our findings: Mean Absolute Error, Mean Squared Error, Root Mean Squared Error, and Mean Absolute Percentage Error. The obtained results are shown in Table 5. The significant outcomes of our approach are illustrated by its predictions, which closely align with the manual annotations of regions of interest and field of view (FOV) lines. Thus, our proposal was successfully deployed into a resource-constrained device, typically used in Edge Computing applications, providing significant results in detecting duplicated subjects in multi-camera monitoring without requiring high-quality recordings or user identification.

After fine-tuning the best model in the indoor environment, we evaluated the performance of our proposal in real-world scenarios supporting the BRT system. In Fig. 3, we illustrate this application, including the FOV line (right-most image) generated by our approach. Consistent with the previous results, Fig. 15a and b highlight the performance of our method in a given station within a specific time interval. The strong performance of our solution underscores its significance, while the mean discrepancies in the real count (Fig. 15a) are attributed to image quality issues, as the system may not detect passengers located far from the camera. In summary, our approach

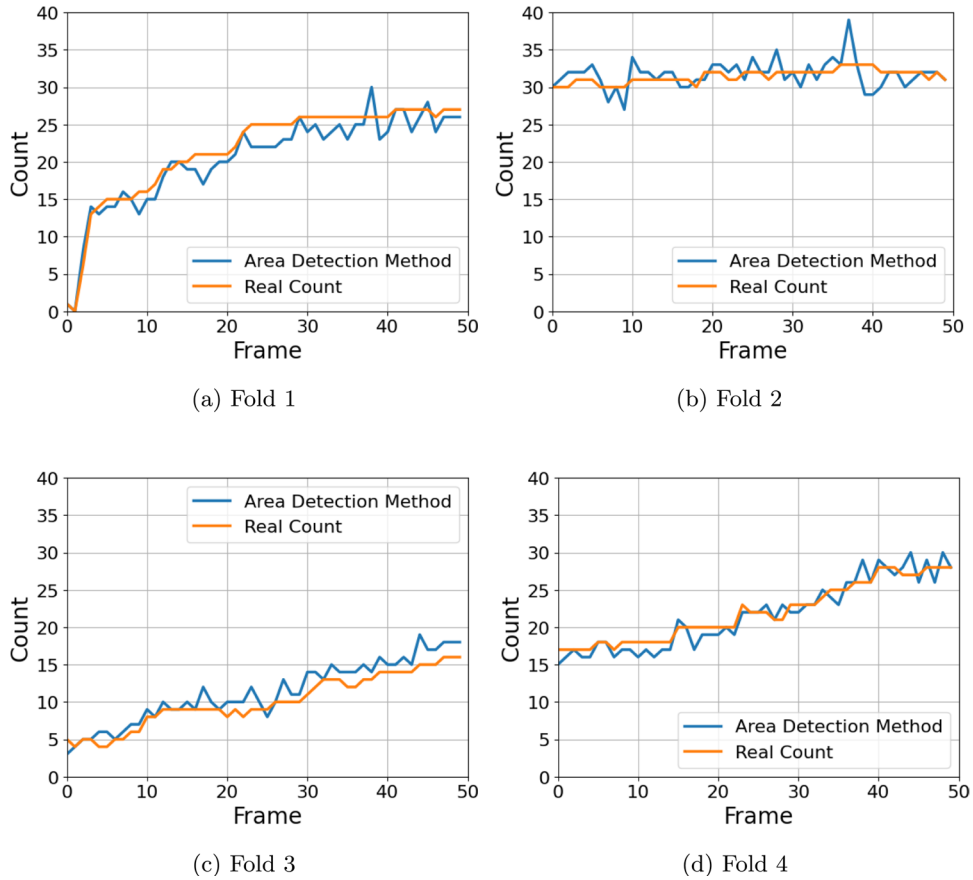


Fig. 12 Comparison between the real person count and counting made using the Area Detection Approach

provided MAE = 1.220, MAPE = 0.255, MSE = 2.820, and RMSE = 1.679, and the annotations' metrics were MAE = 0.060, MAPE = 0.005, MSE = 0.100, and RMSE = 0.316. These results confirm the effectiveness and high quality of our proposed method.

5 Final remarks

In this work, we address the challenging task of reducing the impact of duplicated information in multi-camera surveillance systems. This problem arises when the system monitors the same space, creating overlap in regions covered by two or more cameras. In the literature, there are well-known approaches to overcome this situation, such as subjects' re-identification to detect duplicates. However, the performance of such models can be significantly affected by factors like the analyzed target, video quality, and the distance between subjects and cameras.

Our contribution introduces an efficient and adaptable approach for automatically detecting overlapping fields of view (FOV) without relying on high-quality recordings or subject re-identification. By combining classification and regression methods, our method significantly reduces the need for detailed subject features, making it suitable for both indoor and outdoor environments and especially valuable in resource-constrained devices, such as those used in edge computing. The results demonstrate that our method achieves performance levels comparable to manually labeled models, while maintaining flexibility: if monitoring cameras are repositioned, the system automatically recalculates the FOV lines, ensuring adaptability to dynamic environments.

As highlighted in our experiments, yolo11n was approximately 30 times faster than yolo11x and achieved a higher F1-score after fine-tuning with our approach than all versions of YOLOv11 without fine-tuning, reinforcing the efficiency and accuracy of our detection pipeline. Besides, an additional evaluation was conducted by

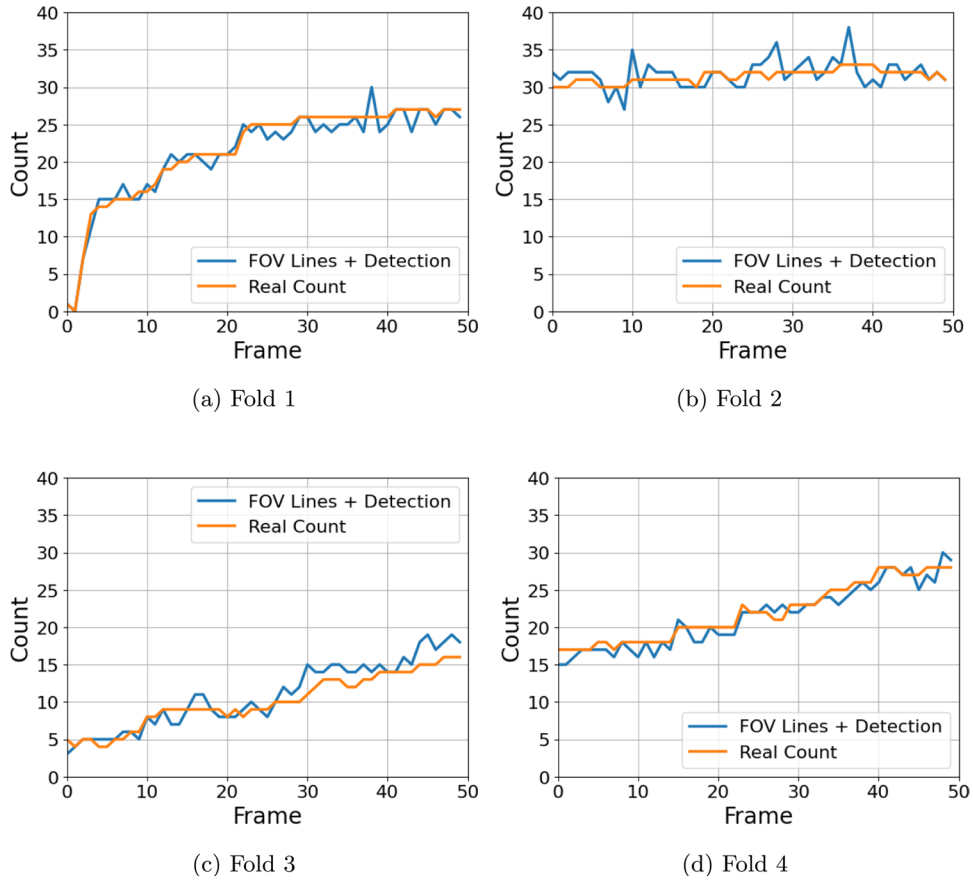


Fig. 13 Comparison between the real person count and counting made using the estimated FOV lines and the detections from our model

using the estimated FOV lines to count manually annotated users instead of relying on the outputs of the face detection model. As shown in our results, this analysis revealed that some counting errors were due to inaccuracies in user detection rather than in FOV estimation. This suggests that our approach could yield even more accurate results when integrated with a more precise user detection model, further demonstrating the robustness and potential of our proposed solution.

As future work, we plan to evaluate our approach using alternative regression methods. Although wRANSAC offers a simple and effective solution, it operates as a linear model to detect overlapping regions. However, depending on the spatial distribution of subjects, a non-linear mapping may yield better results, particularly as the number of monitored cameras increases. Another promising avenue for future work is the adoption of weak supervision or semi-supervised learning, which can markedly reduce labeling effort, especially in non-stationary settings where class patterns evolve over time. To this end, a lightweight face detector can propose candidate boxes, high-confidence detections can be retained with temporal smoothing, and object trackers can propagate boxes across frames. An active-learning loop can include a human in the loop to resolve uncertain cases. For FOV boundaries, candidate lines can be estimated from motion-density heatmaps and optical-flow discontinuities, followed by targeted spot checks where uncertainty remains high. This strategy can substantially reduce annotation costs while preserving accuracy.

Our pipeline currently assumes that a subject crossing the edge will reappear in the adjacent camera, an assumption that may not hold under severe occlusion or complex multi-camera geometry. As future work, this handoff can be generalized from a deterministic rule to a probabilistic mechanism that fuses scene topology and tracklet evidence, improving robustness across layouts. This can be achieved by adding short-horizon trajectory extrapolation to bridge missed detections during occlusions. As discussed by Liu et al. [25], single-frame location anticipation has shown that subsequent locations can be predicted from a static keyframe by exploiting smooth

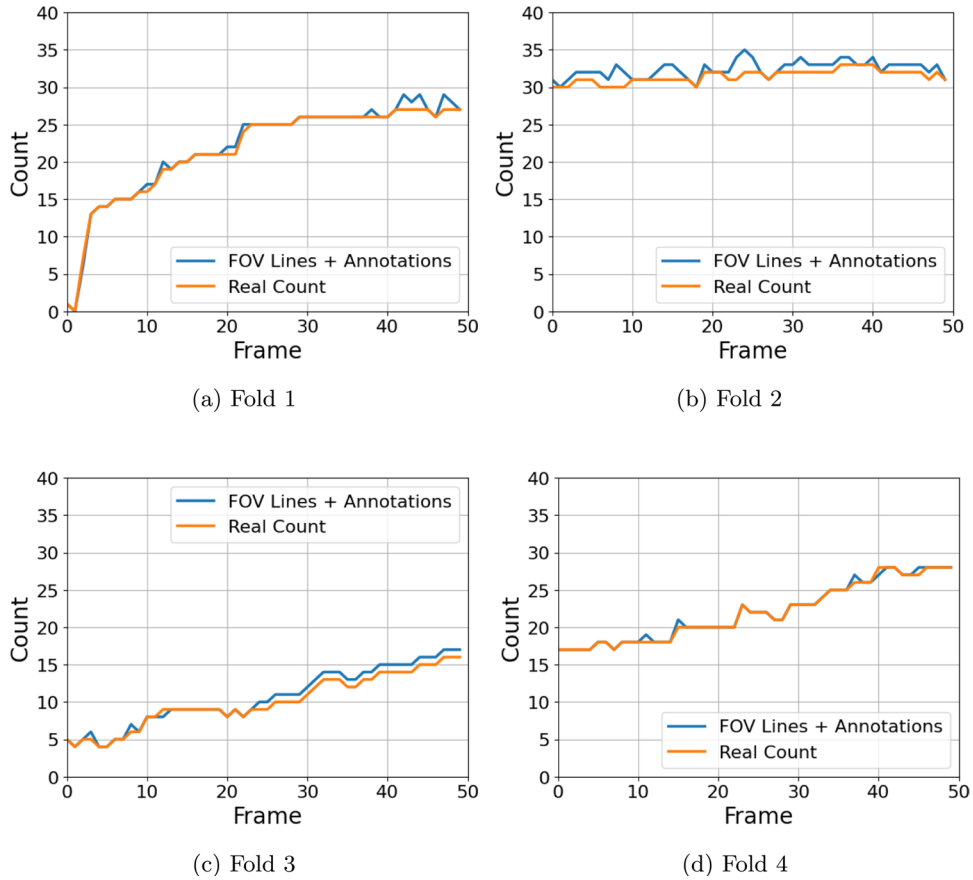


Fig. 14 Comparison between the real person count and counting made using the estimated FOV lines on manual annotations

Table 5 Validation metrics for all experiments: area detection, our approach, and manual annotations

Supervised integration: area detection				
Fold	**MAE**	**MAPE**	**MSE**	**RMSE**
1	1.520	0.072	3.480	1.865
2	1.280	0.041	3.080	1.755
3	1.340	0.141	2.740	1.655
4	0.980	0.046	1.620	1.273
Our approach + face detection				
Lecture	**MAE**	**MAPE**	**MSE**	**RMSE**
1	0.900	0.042	1.660	1.288
2	1.280	0.041	2.920	1.709
3	1.260	0.123	2.660	1.631
4	0.920	0.043	1.400	1.183
Our approach + manual annotations				
Lecture	**MAE**	**MAPE**	**MSE**	**RMSE**
1	0.300	0.015	0.420	0.648
2	0.940	0.030	1.540	1.241
3	0.580	0.051	0.580	0.762
4	0.100	0.004	0.100	0.316

motion. Adapting this idea allows us to keep tracklets alive through occlusions and postpone duplicate suppression until the predicted reappearance window expires, which directly targets the observed failure mode.

Finally, while we explored a classical face-based re-identification approach, the lower performance highlights the need to extend our pipeline with domain adaptation and anchor-free detection. For re-ID, promising directions

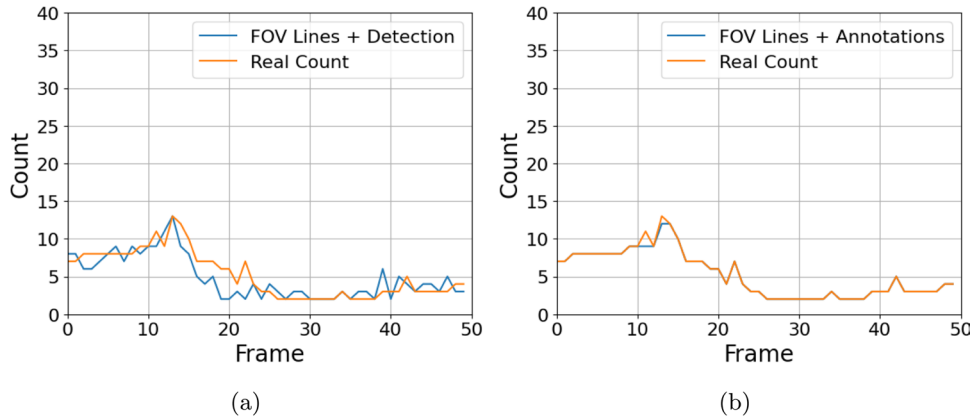


Fig. 15 **a** Comparison between the real person count and counting made using the estimated FOV lines on detections from the model. **b** Comparison between the real person count and counting made using the estimated FOV lines on manual annotations

include lightweight backbones with unsupervised adaptation to new camera domains, leveraging pseudo-labels and temporal consistency to reduce supervision while respecting resource constraints. For detection and tracking, adopting anchor-free heads that directly predict object centers and sizes can simplify hyperparameter tuning and reduce latency on edge devices. Together, these extensions aim to improve robustness across layouts and illumination conditions, including viewpoint and scale changes, while preserving the efficiency that motivates our deployment.

Acknowledgements This work was supported by CNPq (Brazilian National Council for Scientific and Technological Development), UFBA/CNPq 68/2022 - MAI/DAI, Maria Emilia Foundation grant to 01/2023, INCITE FAPESB (Bahia Research Foundation) grant TO PIE0002/2022, and CAPES (Coordination for the Improvement of Higher Education Personnel -- Brazil) Finance Code 001.

Author Contributions All authors contributed to the conception and design of the study. Material preparation and data collection were carried out by Jorge Batista, Matheus Guimarães, and Jorge Nery. Model and method design were conducted by Jorge Batista. All authors read and approved the final manuscript.

Funding The Article Processing Charge (APC) for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614).

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Elharrouss O, Almaadeed N, Al-Maadeed S (2021) A review of video surveillance systems. J Vis Commun Image Represent 77:103116
2. Sultani W, Chen C, Shah M (2018) Real-world anomaly detection in surveillance videos. In: 2018 IEEE/CVF conference on computer vision and pattern recognition, pp 6479–6488. https://doi.org/10.1109/CVPR.2018.00678
3. Yang H, Jang H, Kim T, Lee B (2019) Non-temporal lightweight fire detection network for intelligent surveillance systems. IEEE Access 7:169257–169266. https://doi.org/10.1109/ACCESS.2019.2953558

4. Kanagamalliga S, Kovalan P, Kiran K, Rajalingam S (2023) Enhancing traffic surveillance and urban mobility with vision-based vehicle analysis. In: 2023 international conference on system, computation, automation and networking (ICSCAN), pp 1–6. https://doi.org/10.1109/ICSCAN58655.2023.10394964
5. Cheng JP, Wong PK-Y, Luo H, Wang M, Leung PH (2022) Vision-based monitoring of site safety compliance based on worker re-identification and personal protective equipment classification. Autom Constr 139:104312
6. Liu J, Gu Y, Kamijo S (2017) Customer behavior classification using surveillance camera for marketing. Multimedia Tools Appl 76(5):6595–6622. https://doi.org/10.1007/s11042-016-3342-1
7. Li J, Huang L, Liu C (2012) People counting across multiple cameras for intelligent video surveillance. In: 2012 IEEE 9th international conference on advanced video and signal-based surveillance, pp 178–183. https://doi.org/10.1109/AVSS.2012.54
8. Wang S, Li R, Lv X, Zhang X, Zhu J, Dong J (2018) People counting based on head detection and reidentification in overlapping cameras system. In: 2018 international conference on security, pattern analysis, and cybernetics (SPAC), pp 47–51. https://doi.org/10.1109/SPAC46244.2018.8965468
9. Cheng Z, Zhu X, Gong S (2018) Surveillance face recognition challenge
10. Xu X, Liu W-Q, Li L (2014) Low resolution face recognition in surveillance systems. J Comput Commun 2:70–77
11. Lu Z, Jiang X, Kot A (2018) Deep coupled resnet for low-resolution face recognition. IEEE Signal Process Lett 25(4):526–530. https://doi.org/10.1109/LSP.2018.2810121
12. Yu H-X, Wu A, Zheng W-S (2018) Unsupervised person re-identification by deep asymmetric metric embedding. IEEE Trans Pattern Anal Mach Intell 42(4):956–973
13. Lowe DG (2004) Distinctive image features from scale-invariant keypoints. Int J Comput Vis 60:91–110
14. Bay H, Tuytelaars T, Van Gool L (2006) Surf: speeded up robust features. In: Computer Vision–ECCV 2006: 9th European conference on computer vision, Graz, Austria, May 7-13, 2006. Proceedings, Part I 9, pp 404–417. Springer
15. Rublee E, Rabaud V, Konolige K, Bradski G (2011) Orb: an efficient alternative to sift or surf. In: 2011 International conference on computer vision, pp 2564–2571. IEEE
16. Suttichaya V, Cherdchusakulchai R, Phimsiri S, Trairattanapa V, Tungjitnob S, Kudisthalert W, Kiawjak P, Thamwiwatthana E, Borisuitsawat P, Tosawadi T, et al (2024) Online multi-camera people tracking with spatial-temporal mechanism and anchor-feature hierarchical clustering. In: Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), vol 7
17. Kim J, Shin W, Park H, Baek J (2023) Addressing the occlusion problem in multi-camera people tracking with human pose estimation. In: Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), pp 5463–5469
18. Nguyen QQ-V, Le HD-A, Chau TT-T, Luu DT, Chung NM, Ha SV-U (2023) Multi-camera people tracking with mixture of realistic and synthetic knowledge. In: Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), pp 5496–5506
19. Yoshida R, Okubo J, Fujii J, Amakata M, Yamashita T (2024) Overlap suppression clustering for offline multi-camera people tracking. In: Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), pp 7153–7162
20. Guan Z, Wang Z, Zhang G, Li L, Zhang M, Shi Z, Jiang N (2025) Multi-object tracking review: retrospective and emerging trend. Artif Intell Rev 58(8):1–46
21. Redmon J, Divvala S, Girshick R, Farhadi A (2016) You only look once: unified, real-time object detection. In: Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)
22. Fischler MA, Bolles RC (1981) Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. Commun ACM 24(6):381–395
23. RangiLyu (2021) NanoDet-Plus: Super fast and high accuracy lightweight anchor-free object detection model. https://github.com/RangiLyu/nanodet
24. Serengil S, Ozpinar A (2024) A benchmark of facial recognition pipelines and co-usability performances of modules. J Inform Tech 17(2):95–107. https://doi.org/10.17671/gazibtd.1399077
25. Liu X, Nejadasl FK, Gemert JC, Booij O, Pintea SL (2023) Objects do not disappear: video object detection by single-frame object location anticipation. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 6950–6961

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Jorge Batista[1] · Tatiane N. Rios[1] · Matheus Guimarães[1] · Jorge Nery[1] · Cassio V. S. Prazeres[1] · Rubisley Lemes[2] · Maycon L. Peixoto[1] · Gustavo B. Figueiredo[1] · Frederico A. Durão[1] · Eduardo S. Almeida[1] · Ivan C. Machado[1] · Hérsio M. Iwamoto[2] · Ricardo A. Rios[1]

✉ Ricardo A. Rios
ricardoar@ufba.br

Jorge Batista
jorgebatista@ufba.br

Tatiane N. Rios
tatiane.nogueira@ufba.br

Matheus Guimarães
guimaraes.matheus@ufba.br

Jorge Nery
jorgenery@ufba.br

Cassio V. S. Prazeres
prazeses@ufba.br

Rubisley Lemes
rubisley@ufba.br

Maycon L. Peixoto
maycon.leone@ufba.br

Gustavo B. Figueiredo
gustavobf@ufba.br

Frederico A. Durão
fdurao@ufba.br

Eduardo S. Almeida
esa@ufba.br

Ivan C. Machado
ivan.machado@ufba.br

Hérsio M. Iwamoto
hersio@positivo.com.br

1 Institute of Computing, Federal University of Bahia (UFBA), Av. Milton Santos, s/n, Salvador, BA 40170-110, Brazil

2 Positivo, Curitiba, PR, Brazil